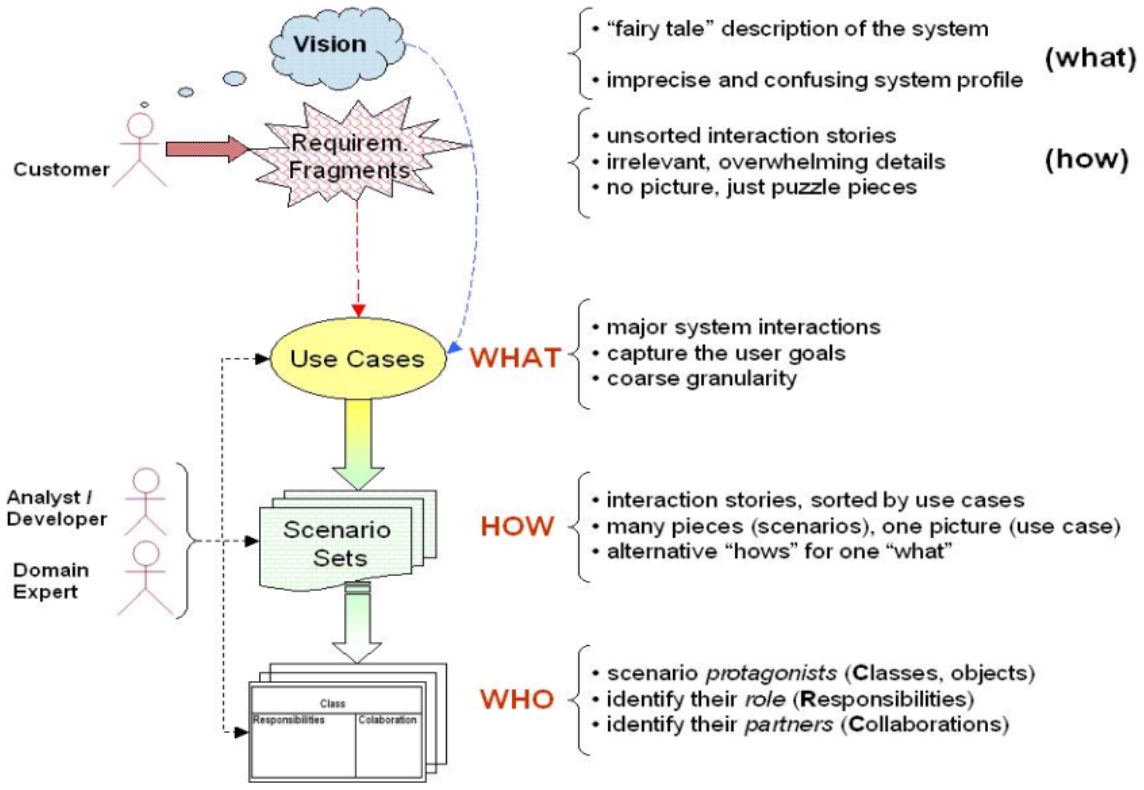
Lecture-3

Requirements Engineering in a Nutshell



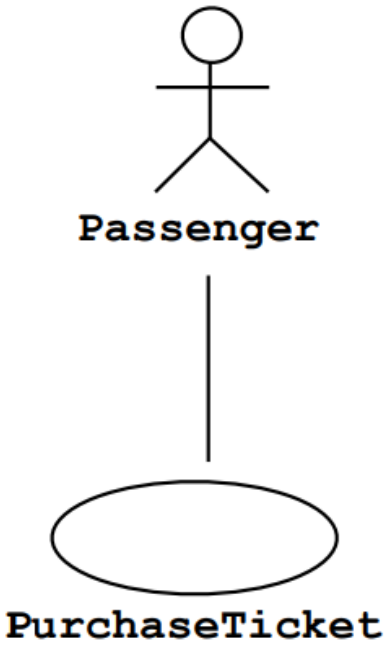
➢ Cum sa construim un sistem? Avem nevoie de o pereche de roluri : nevoie de intelegere a domeniului aplicatiei (un business man care sa puna in ordine viziunea clientului, sa preia elementele care reprezinta acele goluri din viziunea lui si sa le defineasca intr-un mod coerent), dar si de o nevoie de intelegere a partii tehnice (un specialist cre sa inteleaga cum se pot transforma aceste lucruri intr-un sistem SW) -> primele doua acolade
➢ Ce sa faca sistemul si cum sa faca sistemul? E nevoie in prima faza sa vedem care vor fi interactiunile dintre utilizatori si sistemul in sine -> SUPER IMPORTANT -> a treia acolada
➢ Pentru intrebarea cum se va manifesta sistemul : ex, am un editor de poze -> features : poate mofica parametrii de imagine, poate reduce zgomote (bucati mari de functionalitae -> usecase□uri) etc. ; mereu am un CUM pentru un CE
➢ Putem avea mai multe scenarii pt un usecase -> urmatorul pas e sa scriem cod -> pt fiecare usecase sa identificam clasele si obiectele, responsabilitatile si interactiunile intre clase
➢ Cand pornim sa facem usecase-uri, avem un mod systematic : cine sunt utilizatorii sistemului (actorii) ? -> un usecase serveste un actor al sistemului -> cine interactioneaza cu sistemul? -> care sunt interactiunile fiecaruia dintre actorii sistemului? -> pt fiecare usecase, cum se va desfasura el? Sunt mai multe moduri/scenarii.

Use Cases

- create a set of **scenarios** that identify a thread of usage
  - for the system to be constructed
- scenarios are often called use-cases [Jacobson 1992]
  - how systems will be used

➢ Avantaje : este sistematic si am un process ierarhic(am putini actori -> roluri diferite , fiecare are un nr limitat de usecase-uri -> pot sa vad cum interactioneaza cu sistemul)
➢ Pt fiecare usecase defines un set de scenarii -> ca sa imi spuna un mod de utilizare al sistemului -> au insemnatate pt utilizatori(actori)

Use case diagrams



Used during requirements elicitation to represent external behavior

- ***Actors*** represent roles,
  - a type of user of the system
- ***Use cases*** represent a sequence of interaction for a type of functionality
- ***Use case model***
  - the set of all use cases.
  - a complete description of the functionality of the system and its environment

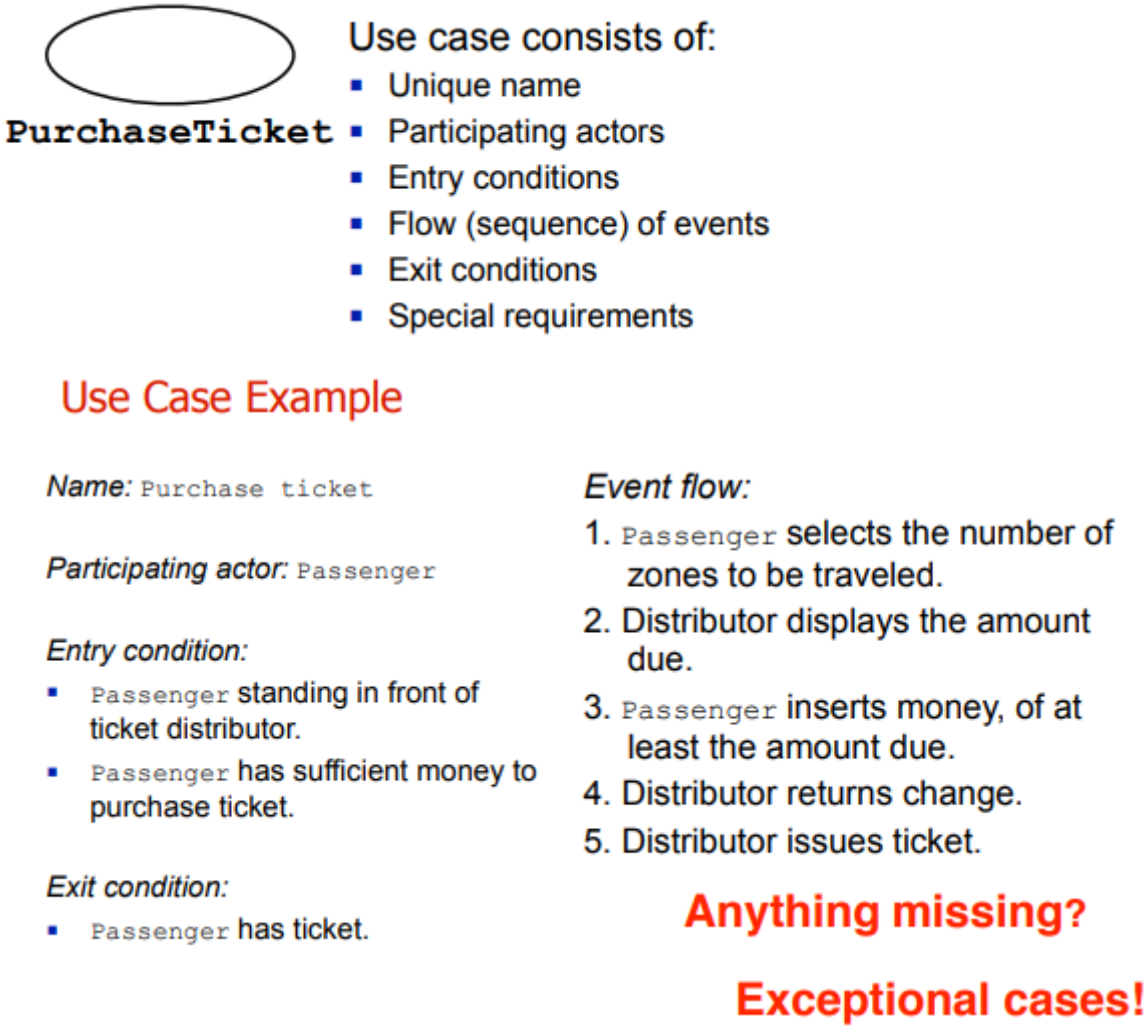
Actors

```
➢ Actorii sunt entitati externe sistemului care comunica cu sistemul-> nu
trebuie sa fie o pers fizica, un om -> poate fi un om sau un alt sistem care
comunica cu acest sistem sau un mediu fizic
➢ Un actor are un nume unic si o descriere optionala
➢ EX : Passenger : A person in the train
➢ EX2: GPS satellite: Provides the system with GPS coordinates
➢ Daca un sistem comunica cu alt sistem si doar schimba date si nu ii poate
vedea codul celuilalt sistem spunem ca acel sistem este extern
➢ Actorii nu fac parte din sistem -> actorul poate sa fie alt sistem, dar
```

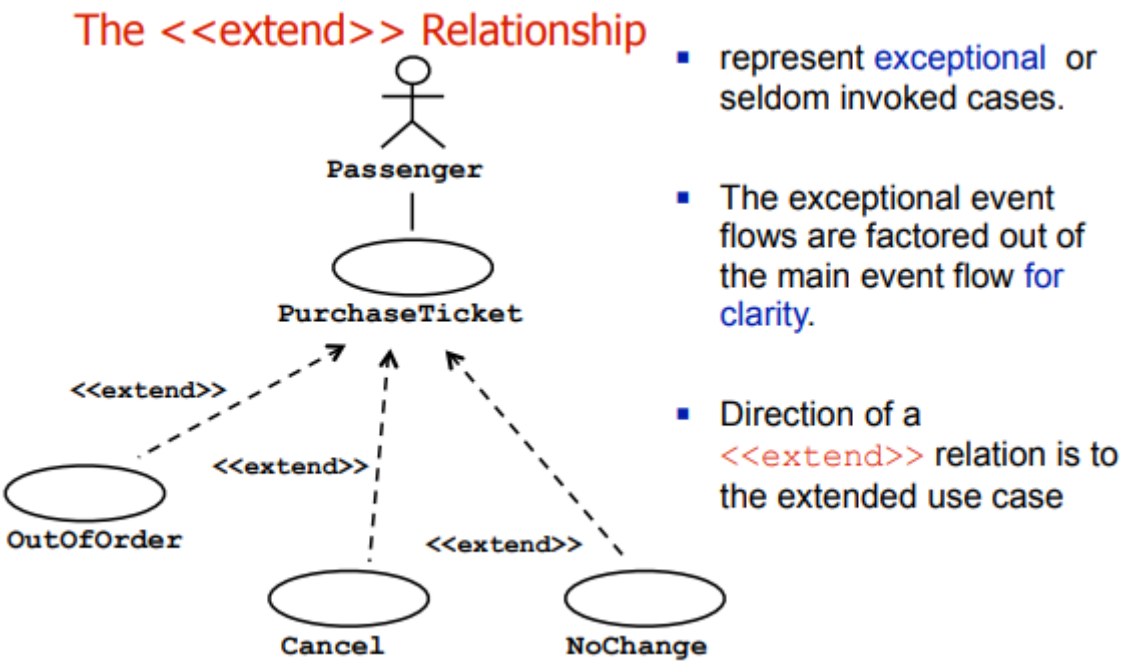
```
nu poate fi o parte din sistemul pe care deja il modelezi
➢ Actorul e mereu cineva care interactioneaza cu sistemul : poate ori
introduce,ori primi informatii din sistem ori ambele
➢ Spre ex, am doua persoane Ale si Ioana care lucreaza la primarie care
interactioneaza cu sistemul si vor sa inregistreze masini -> nu voi avea 2
actori , ci voi avea unul singur, pentru ca ambele persoane au acelasi rol -
> Acela de a introduce masini (chiar daca ele sunt entitati diferite, pt
sistem ele vor reprezenta doar o entitate) -> daca era vreo diferenta legata
de modul in care acestia introduce masini, atunci da, se considera ca erau 2
actori
➢ De ex, am un actor professor si am un actor student -> am un al treilea
actor doctorand? -> nu, pt ca el poate juca si rolul de professor(preda si
el), dar si rolul de student(inca e la doctorat)
```

Other info on Use Case

```
➢ Usecase-ul e o bucata mare de functionalitate -> are multe scenarii , dar
unul e default
```

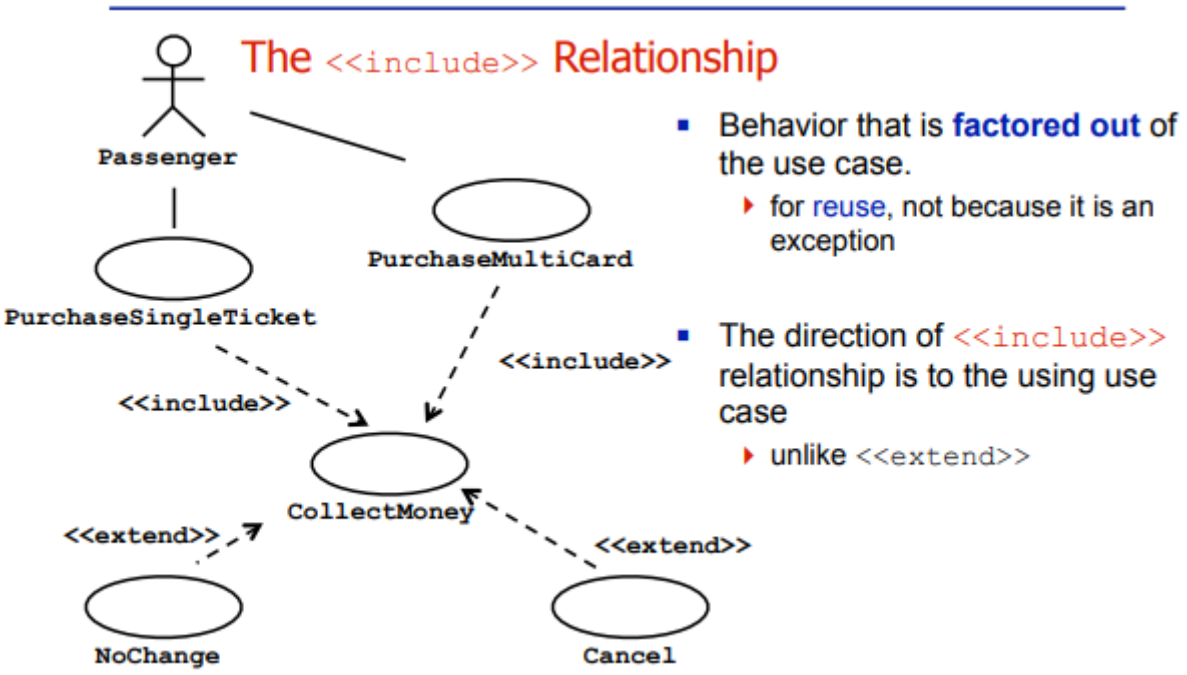


The extend Relationship



Sageata de la extend e de la cazul exceptional catre cazul de baza

The include Relationship



Sageata e invers. PurchaseSingleTicket include CollectMoney

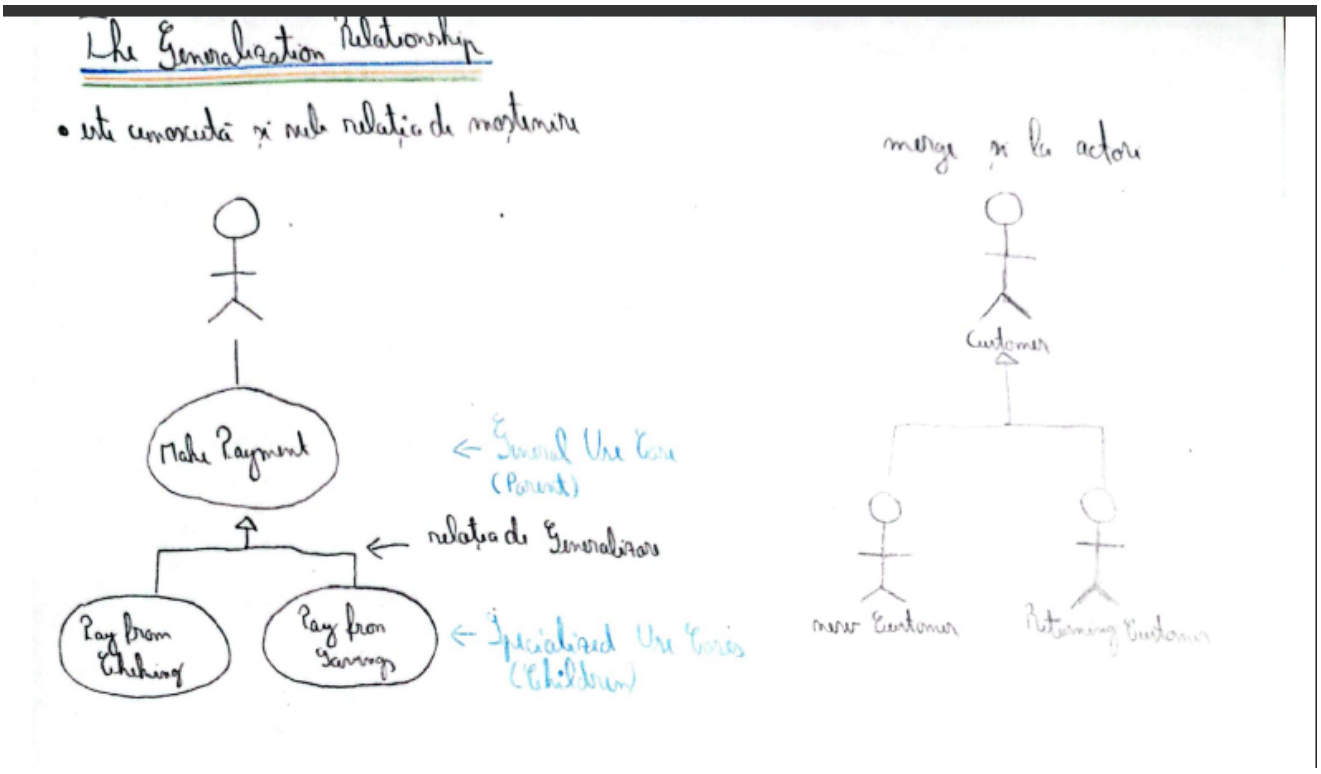
Use Case Modeling : Core Elements

Construct	Description	Syntax
use case	A sequence of actions, including variants, that a system (or other entity) can perform, interacting with actors of the system.	UseCaseName
actor	A coherent set of roles that users of use cases play when interacting with these use cases.	ActorName
system boundary	Represents the boundary between the physical system and the actors who interact with the physical system.	

Use Case Modeling : Core Relationships

Construct	Description	Syntax
association	The participation of an actor in a use case. i.e., instance of an actor and instances of a use case communicate with each other.	
generalization	A taxonomic relationship between a more general use case and a more specific use case.	
extend	A relationship from an *extension* use case to a *base* use case, specifying how the behavior for the extension use case can be inserted into the behavior defined for the base use case.	<<extend>>

Construct	Description	Syntax
include	An relationship from a *base* use case to an *inclusion* use case, specifying how the behavior for the inclusion use case is inserted into the behavior defined for the base use case.	<<include>>



Three Levels of Abstractions for Use Cases

1. Sea Level (cele adevarate, reprezinta o interactiune majora a actorilor cu sistemul)
    1. Users interacts with system
    2. Major interactions
    3. Precise goal for using a system
2. Fish Level (toate include urile, extend urile, scenariile)
    1. the use cases that factor out commonalities
3. Kite Level (apare la sistemele/ companiile foarte mari)
    1. show how sea-level cases fit into wider business interactions
    2. cazuri absolut exceptionale

Identifying Actors

Actorii nu sunt parte din sistem (ei interactioneaza cu sistemul din afara)
Un actor ar putea :
1. Da informatie de intrare sistemului
2. Sa primeasca informatie de la sistem
3. Ambele

- Don't create an actor based on the official position of a person
    - i.e. if a person fulfills a combination of roles, each role is an actor, but the combination is not!

A use case typically represents a major piece of functionality that is complete from beginning to end.
A use case must deliver something of value to an actor.

- Good Questions:
  - Can all functional requirements be performed by the use cases?
  - What are the tasks of each actor?

Use case modeling tips

- Each use case describes a significant chunk of system usage
  - that is understandable by both domain experts and programmers
- When defining use cases in text, use nouns and verbs accurately and consistently
  - to help derive later objects and messages for interaction diagrams
- Factor out common usages that are required by multiple use cases
  - If the usage is required use <<include>>
  - If the base use case is complete and the usage may be optional, consider use <<extend>>
- A use case diagram should
  - contain only use cases at the same level of abstraction
  - include only actors who are required

Software Requirements Analysis

- Build models of data and behavioral domain
  - that will be treated by software
- Result: representation of data, function
  - behavior that can be translated into design:
- Means to assess quality once the system is built

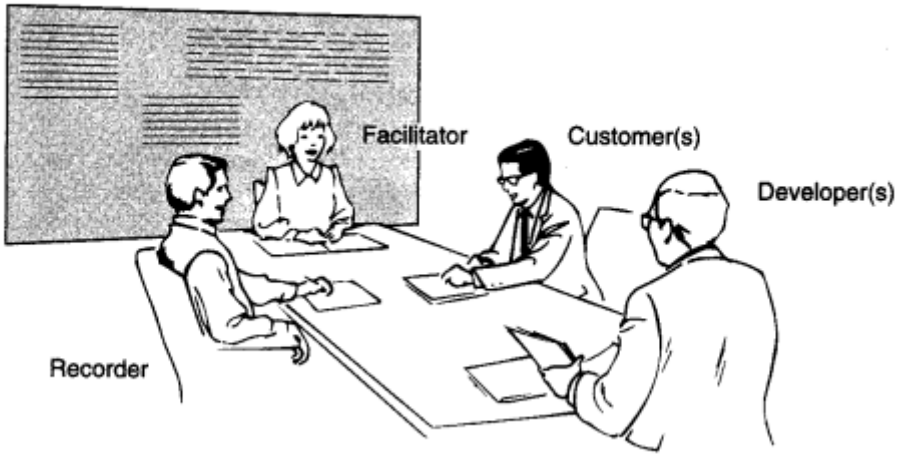
Initiate the Requirements Analysis Process

- First Meetings like ... dating :)
  - both want it to be a success, yet they don't know what to say and are afraid that it might be misinterpreted
- Ask context-free questions
  - Who requested this work? Who will use the solution? What's the benefit?
  - What is a good output? What's the environment where it will work?
- Ask meta-questions
  - Are my question relevant? Should I be asking you something else?
  - Do you feel comfortable with answering my questions? Is there someone else who could answer these questions more easy?

Facilitated Application Specification Techniques (FAST)'

- Defeat the "*us and them*" mentality
  - communicate through more than memos and documents
- **Goal**:
  - to identify problem, specify a preliminary set of requirements

- **Guidelines**:
  - Meeting at neutral site with both sw. engineers and customers
  - Agenda Formal-Informal
    - cover major points, yet stimulate free flow of ideas
  - Facilitator controls the meeting
    - best if it's an outsider



FAST Process

- Start meeting from a 1-2 pg. "*product request*"
  - as a result of initial discussions
- Each participant comes to the meeting with **4 lists:**
  1. List of **Objects**
     a. produced objects
     b. used objects, during processing
     c. environmental objects (surround the system)
  2. List of **Services**
     - functions and processes that manipulate the objects
  3. List of **Constraints**
     - cost, size, business rules
  4. List of **Performance Criteria**
     - speed, accuracy
- Lists don't have to be exhaustive, just reflect various p.o.v

Fiecare din cei 4 oameni vine cu 4 liste sumare

- Start with an agreement on the justification of the product
- Pin lists on wall
- Create a combined list on each topic area
  - eliminate redundancies
  - **don't delete anything!**
- Develop a consensus list
  - after all lists have been combined...
  - lists get shortened, lengthened, rephrased
- Divide team into sub-teams and develop mini-specifications
  - Alternative: develop Use Cases
- Joined meeting and discuss emerged changes
  - also build a *list of issues*